Lecture 6 – Circular Dependency, Bean Scope & Bean Initialization – Quick Revision

1. Quick Recap: DI, IoC, Bean

- **DI** = Spring gives the dependency to a class instead of class creating it itself.
- **IoC** = Control of object creation goes to Spring container.
- **Bean** = An object created & managed by Spring container.

2. @Configuration

`@Configuration` classes internally have `@Component` too, so Spring auto-detects them. They can hold `@Bean` methods to create beans.

3. Bean Creation Order

Spring doesn't create beans randomly — it creates dependencies first, then the class that needs them (e.g., PaymentGateway → PaymentService → OrderService → OrderController).

4. Why Order Matters for Circular Dependency

If A needs B and B needs A, Spring gets stuck in a loop trying to decide which to create first — this is the root cause of circular dependency.

5. What is Circular Dependency

Two or more classes depend on each other (directly or indirectly). Example: OrderService needs PaymentService, PaymentService needs OrderService.

6. Not Just a Spring Problem

Even plain Java has this issue — two classes calling `new` on each other in constructors causes infinite recursion → `StackOverflowError`. Spring just exposes this during bean creation.

7. Why Constructor Injection Fails in Circular Dependency

Constructor injection rule: object can't be created until ALL constructor args are ready. If A needs B and B needs A in constructors, there's no starting point → Spring throws `BeanCurrentlyInCreationException`, app fails to start.

8. Constructor Injection Still Recommended

Even though it fails on circular dependency, constructor injection is preferred because it

makes dependencies clear & mandatory. The real problem is circular design, not constructor injection itself.

9. Setter Injection & Circular Dependency

In setter injection, object creation and dependency injection are separate steps — so Spring CAN create both objects first, then inject them into each other later. This may resolve some circular dependencies.

10. Field Injection & Circular Dependency

Same idea as setter injection — object created first, dependency (`@Autowired` field) injected later, so Spring may handle circular dependency here too.

11. Why Setter/Field Injection May Work

Constructor needs dependency immediately; setter/field don't. So Spring can create an "empty" object first and fill dependencies afterward — this flexibility is what enables resolving circular deps.

12. Early Reference Concept

Spring can expose a reference to a bean before it's FULLY initialized, so another bean can temporarily use it while circular dependency is being resolved. This is how Spring's internal circular-dependency resolution works.

13. Spring Boot Discourages Circular References

From Spring Boot 2.6+, circular references are **disabled by default** (`spring.main.allow-circular-references=false`). Can be manually enabled but NOT recommended — treat as last resort, not a solution.

14. Circular Dependency = Design Smell

Usually means responsibilities aren't separated properly. Fix by: moving shared logic to a third class, splitting responsibilities, or using interfaces/events instead of direct two-way dependency.

15. Bean Scope – What is it?

Bean scope decides **how many objects** Spring creates for one bean definition and how long they live. Main scopes: `singleton`, `prototype`; web scopes: `request`, `session`, `application`, `websocket`.

16. Singleton Scope

Default scope. Spring creates **ONE object per bean definition** in the container and reuses it every time. NOT the same as "only one object of this class can exist" — you can still manually do `new`.

17. Singleton is Per Bean Definition, Not Per Class

If you have two `@Bean` methods creating the same class, each is a SEPARATE bean definition → each gets its own singleton object. So "singleton" = one per bean name, not one per class.

18. Prototype Scope

Spring creates a **NEW object every time** the bean is requested from the container. Closer to using `new` manually, but Spring still manages creation.

19. When to Use Singleton vs Prototype

- Singleton → stateless beans with behavior (services like `PaymentService`).
- Prototype → stateful beans that hold changing/request-specific data.

20. Prototype Injected into Singleton

If a prototype bean is injected into a singleton, it does NOT get a fresh prototype object every time — only ONE prototype object is injected at the time singleton is created and reused after that.

21. Web Scopes

- `request` → one object per HTTP request
- `session` → one object per user session
- `application` → one object for whole web app (tied to ServletContext)
- `websocket` → one object per websocket session (Only available in web-aware Spring context.)

22. Bean Initialization – Eager vs Lazy

- **Eager** = bean created at app startup (default for singleton beans).
- **Lazy** = bean created only when actually needed/requested (`@Lazy`).

23. Why Eager is Default

Eager init gives "fail-fast" behavior — config errors are caught at startup, not later when a user hits a feature. That's why Spring prefers it for singletons.

24. Lazy Initialization

Use `@Lazy` on a class to delay its creation until first requested. Spring only stores the bean definition until then.

25. Lazy Bean Needed by an Eager Bean

If an eager singleton needs a lazy bean as dependency, the lazy bean may STILL get created at startup — because Spring must satisfy the eager bean's dependency.

26. @Lazy on Injection Point (Proxy trick)

Instead of marking whole bean lazy, mark the constructor parameter with `@Lazy` — Spring injects a PROXY placeholder immediately, and only creates/resolves the real object when a method on it is actually called.

27. Global Lazy Initialization (Spring Boot)

`spring.main.lazy-initialization=true` makes ALL beans lazy by default (reduces startup time but delays error detection). Default value is `false`. Individual beans can opt out using `@Lazy(false)`.

28. Using @Lazy to Break Circular Dependency

Putting `@Lazy` on one constructor parameter in a circular pair lets Spring inject a proxy instead of the real object immediately — this can "break" the circular startup deadlock. But it's a **workaround**, not a real fix — proper fix is refactoring.

29. Bean Lifecycle (Full Journey)

Bean Definition → Object Creation → Dependency Injection → Initialization → Ready to Use → Destruction.

30. Bean Lifecycle Steps in Detail

1. **Definition** – Spring detects via `@Component`/`@Service`/`@Bean` etc. (just metadata, no object yet)
2. **Creation** – actual object created via constructor
3. **Dependency Injection** – required dependencies injected

4. **Initialization** – setup logic runs (`@PostConstruct`, `InitializingBean`, custom init method)
5. **Ready to Use** – bean is fully usable
6. **Destruction** – on container shutdown, cleanup runs (`@PreDestroy`, `DisposableBean`, custom destroy method) — mainly applies to singleton beans, not prototype.

31. Golden Rules to Remember

- Circular dependency = design smell → fix by refactoring, not hacks.
- Constructor injection fails on circular deps; setter/field injection may survive due to two-step creation.
- Singleton = one object per bean definition (not per class).
- Prototype = new object every time requested from container.
- Eager = fail-fast at startup; Lazy = delays creation/errors until actually used.
- `@Lazy` on injection point = inject proxy now, resolve real bean later.